# Chapter 4: System Design

## 4.1 Introduction

This chapter presents the system design for the Integrated Cyber Threat Detection and Automated Response Platform, translating the functional and non-functional requirements defined in Chapter 3 into a concrete architectural and behavioural design. The chapter is organised into three parts. Section 4.2 presents a black-box view of the platform, showing the system as a single unit together with its external inputs and outputs. Section 4.3 presents the system analysis, using a set of complementary UML diagrams, namely a use-case diagram, a sequence diagram, an activity diagram, an entity-relationship diagram, and a class diagram, to describe who interacts with the system, how those interactions unfold over time, and how the underlying data is structured. Section 4.4 presents the system design proper, culminating in a system architecture diagram that shows how the four platform layers introduced in Chapter 1 (SIEM, honeypot, machine learning engine, and automated response) are physically and logically arranged across the test environment described in Section 3.4.

Together, these diagrams provide the blueprint that Chapter 5 implementation work follows: each component named in the diagrams below corresponds directly to a deployable unit in the five-VM VirtualBox test environment, and each data flow corresponds to a concrete integration point (a Wazuh custom log source, a Wazuh active-response script, or a Python socket/API call) that must be built and validated during the four Agile DSR sprints.

## 4.2 Black-Box Diagram

Figure 4.1 presents the platform as a single black box, abstracting away its internal structure to focus on the boundary between the system and its environment. This view is useful at the start of the design phase because it fixes the contract the system must honour before any internal decomposition is attempted: whatever the internal architecture eventually looks like, it must consume the three input streams shown on the left and produce the three output streams shown on the right.

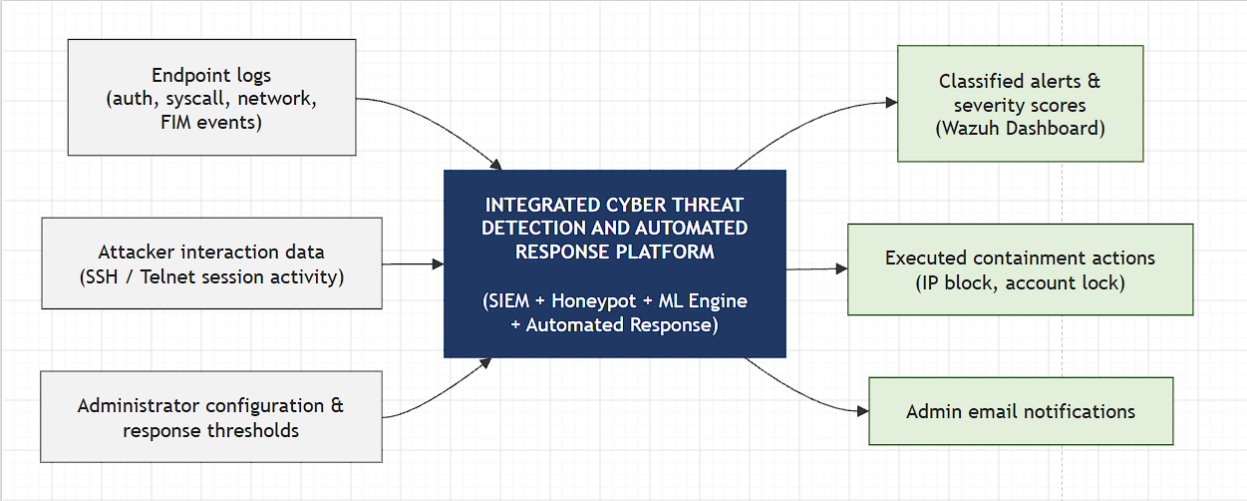



**Figure 4.1: Black-Box Diagram of the Integrated Platform**

On the input side, the platform receives raw endpoint telemetry (authentication logs, system-call activity, network connection events, and file-integrity monitoring data) from the Wazuh agents installed on the monitored Linux endpoints; attacker interaction data captured by the Cowrie honeypot whenever an attacker connects to the decoy SSH or Telnet service; and administrator-supplied configuration, including detection thresholds and response playbook settings. On the output side, the platform produces classified, severity-ranked alerts that are rendered on the Wazuh Dashboard; executed containment actions, namely firewall-level IP blocking and account isolation; and email notifications sent to the designated administrator whenever a critical event is confirmed. The dashed boundary in Figure 4.1 reflects the scope constraint defined in Section 3.3.2: all processing occurs within the isolated VirtualBox internal network, with no dependency on external connectivity during testing.

## 4.3 System Analysis

This section decomposes the black-box view into its constituent actors, behaviours, and data structures using five complementary diagrams. The use-case diagram identifies who or what interacts with the system and for what purpose; the sequence diagram traces one representative interaction (a brute-force attack) through every platform layer in chronological order; the activity diagram generalises this into the full decision logic that governs every detection event, regardless of attack category; the entity-relationship diagram defines the persistent data the platform must

store; and the class diagram defines the object-oriented structure used to implement that behaviour in the Python and Wazuh components built in Chapter 5.

### 4.3.1 Use-Case Diagram

Figure 4.2 identifies three actors that interact with the platform. The Attacker is an external, adversarial human actor represented in testing by the Kali Linux VM, who probes, attacks, and attempts to evade the platform across the seven attack categories defined in Section 1.4. The Security Administrator is the legitimate operator who monitors the dashboard, reviews incidents, and tunes detection rules over time. The ML Engine is modelled as a system actor, a component internal to the platform boundary that nonetheless plays a distinguishable role in the use-case model because it performs autonomous classification decisions that the administrator does not directly control at run time.

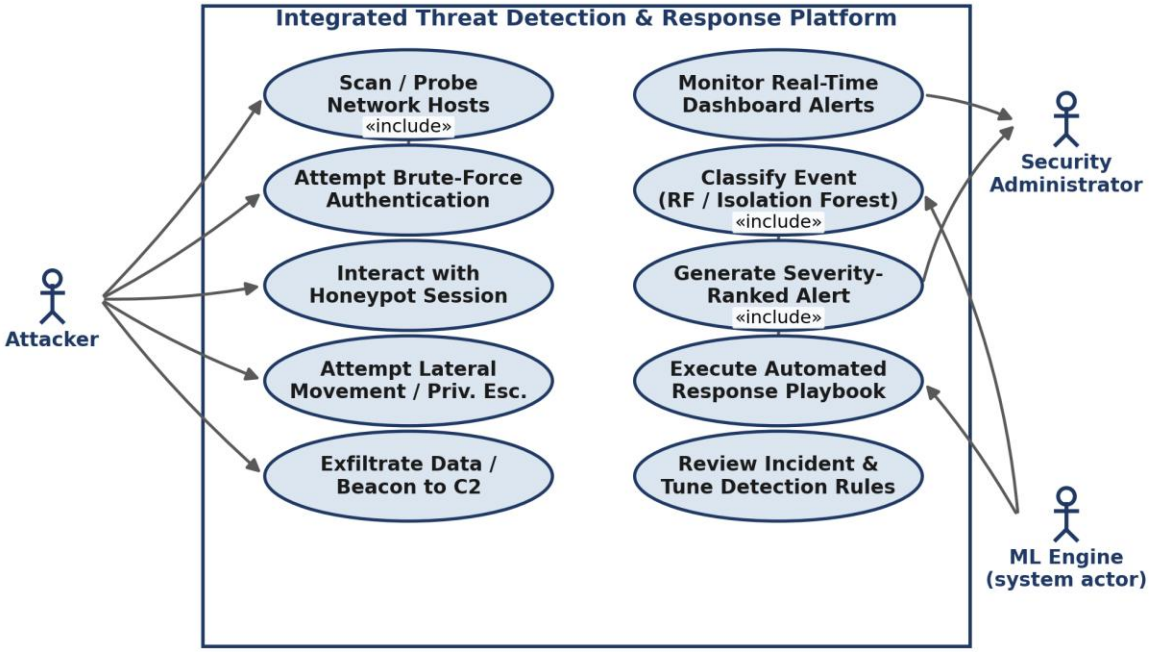



**Figure 4.2: Use-Case Diagram**

Two include relationships are shown. “Attempt Brute-Force Authentication” includes “Scan / Probe Network Hosts” because, in the attack scenarios defined in Section 3.5.3, reconnaissance typically precedes a credential attack. On the defender side, “Generate Severity-Ranked Alert” includes “Classify Event”, since an alert cannot be ranked before the ML Engine has produced a classification, and “Execute Automated Response Playbook” includes “Generate Severity-Ranked Alert”, since the response module only acts on alerts that have already been generated and ranked.

### 4.3.2 Sequence Diagram

Figure 4.3 traces the brute-force detection and response scenario (Scenario 1 in Section 3.5.3) across six participants: the Attacker, the Endpoint / Wazuh Agent, the Wazuh Manager, the ML Engine, the Response Module, and the Administrator. This scenario was chosen for the sequence diagram because it exercises every layer of the platform end to end, from raw log capture through to human acknowledgement, and therefore illustrates the interaction pattern that the remaining six attack categories follow with only the classification label and response playbook changing.

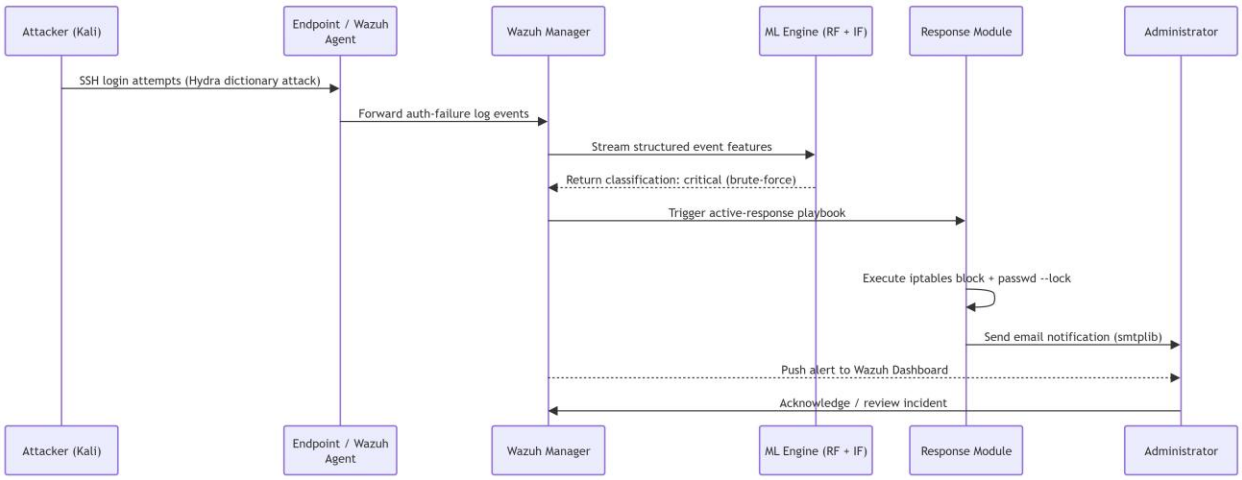



**Figure 4.3: Sequence Diagram; Brute-Force Detection and Response**

The flow begins when Hydra launches a dictionary-based SSH login attack against a monitored endpoint. The Wazuh agent on that endpoint forwards the resulting authentication-failure events to the Wazuh Manager, which streams the extracted event features to the ML Engine. The ML Engine returns a classification of “critical (brute-force)”, at which point the Wazuh Manager triggers the corresponding active-response playbook. The Response Module executes the containment actions, an iptables block and an account lock, and sends an email notification to the administrator using smtplib. In parallel, the Wazuh Manager pushes the alert to the dashboard,

where the administrator reviews and acknowledges the incident. The elapsed time between the first message and the alert appearing on the dashboard corresponds to the Mean Time to Detect metric, and the elapsed time to the executed containment action corresponds to Mean Time to Respond, both defined in Section 3.3.2.

### 4.3.3 Activity Diagram

Figure 4.4 generalises the sequence diagram into the decision logic applied to every event the platform processes, independent of attack category. After the Wazuh agent captures a raw log event and the Wazuh Manager applies its rule-based detection layer, the event features are forwarded to the ML Engine for classification by the Random Forest and Isolation Forest models working in parallel. A decision point then determines whether the event is classified as critical: if not, the event is logged as informational and used to update the platform's behavioural baseline; if it is critical, a severity-ranked alert is generated and passed to the Automated Response Module.

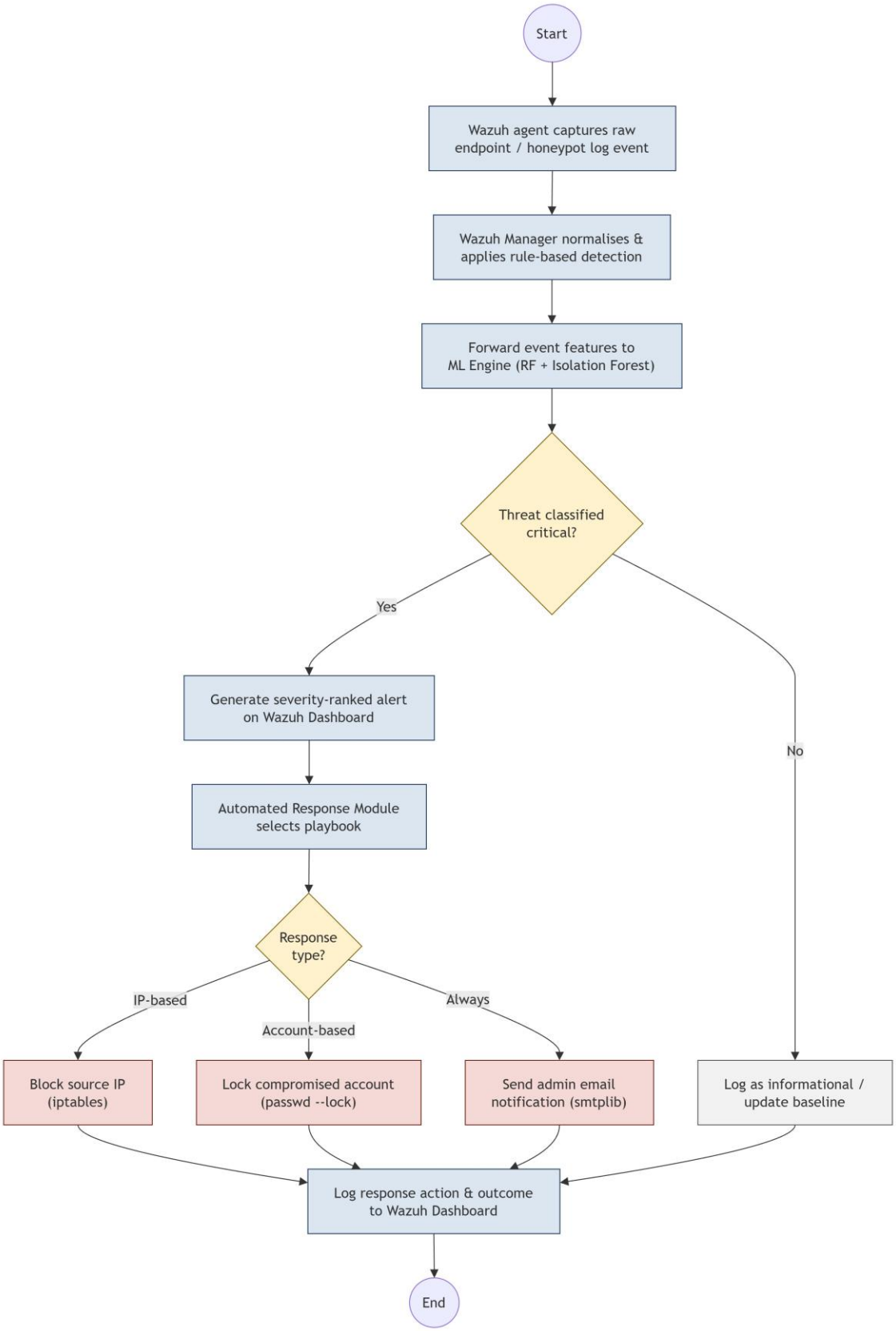



**Figure 4.4: Activity Diagram End-to-End Detection and Response Flow**

A second decision point determines which of the three response playbooks is appropriate to the alert type: IP blocking via iptables for network-based attacks such as port scanning and brute force, account isolation via passwd --lock for compromised-credential and privilege-escalation events, and an administrator email notification, which is sent in every critical case regardless of which containment action is also taken. All three branches converge on a final logging step that records the response action and its outcome back to the Wazuh Dashboard, closing the loop described in the non-functional requirement that response actions be auditable.

### 4.3.4 Entity-Relationship Diagram

Figure 4.5 defines the persistent data model underlying the platform. Eight entities are identified. An Endpoint represents a monitored Linux host; a Honeypot Session represents a single attacker interaction captured by Cowrie. Both generate Log Events, the central entity that every raw event, whether from an endpoint agent or a honeypot session, is recorded against. Each Log Event yields exactly one ML Classification, produced by either the Random Forest or Isolation Forest model, and each classification that meets the critical threshold raises exactly one Alert. Each Alert may trigger one or more Response Actions and is reviewed by an Administrator. An Attack Scenario entity records the five simulated attack scenarios from Section 3.5.3 and is used during validation to check that Log Events generated during testing map back to the scenario that produced them.

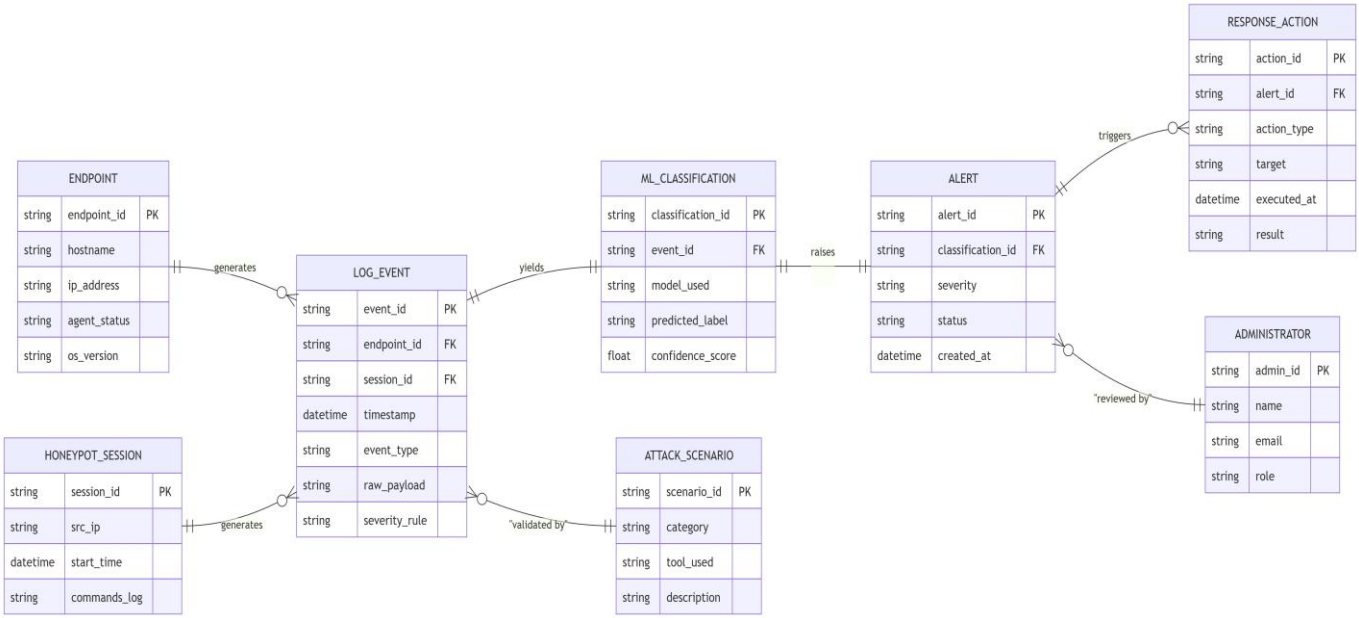



**Figure 4.5: Entity-Relationship Diagram**

The one-to-many relationships shown (an Endpoint generates many Log Events; an Alert may trigger many Response Actions) reflect the platform's real-time, high-volume nature: a single monitored host is expected to generate a continuous stream of events, only a small fraction of which will be classified as critical and raise an alert.

### 4.3.5 Class Diagram

Figure 4.6 translates the entity-relationship model into the object-oriented class structure used to implement the platform's Python components. The LogEvent class carries the normalised event data produced by the WazuhManager class, which is responsible for log collection, rule application, and forwarding events to the MLEngine. The MLEngine aggregates a RandomForestModel and an IsolationForestModel, delegating the supervised and unsupervised classification tasks respectively, and returns a classification that the WazuhManager uses to raise an Alert.

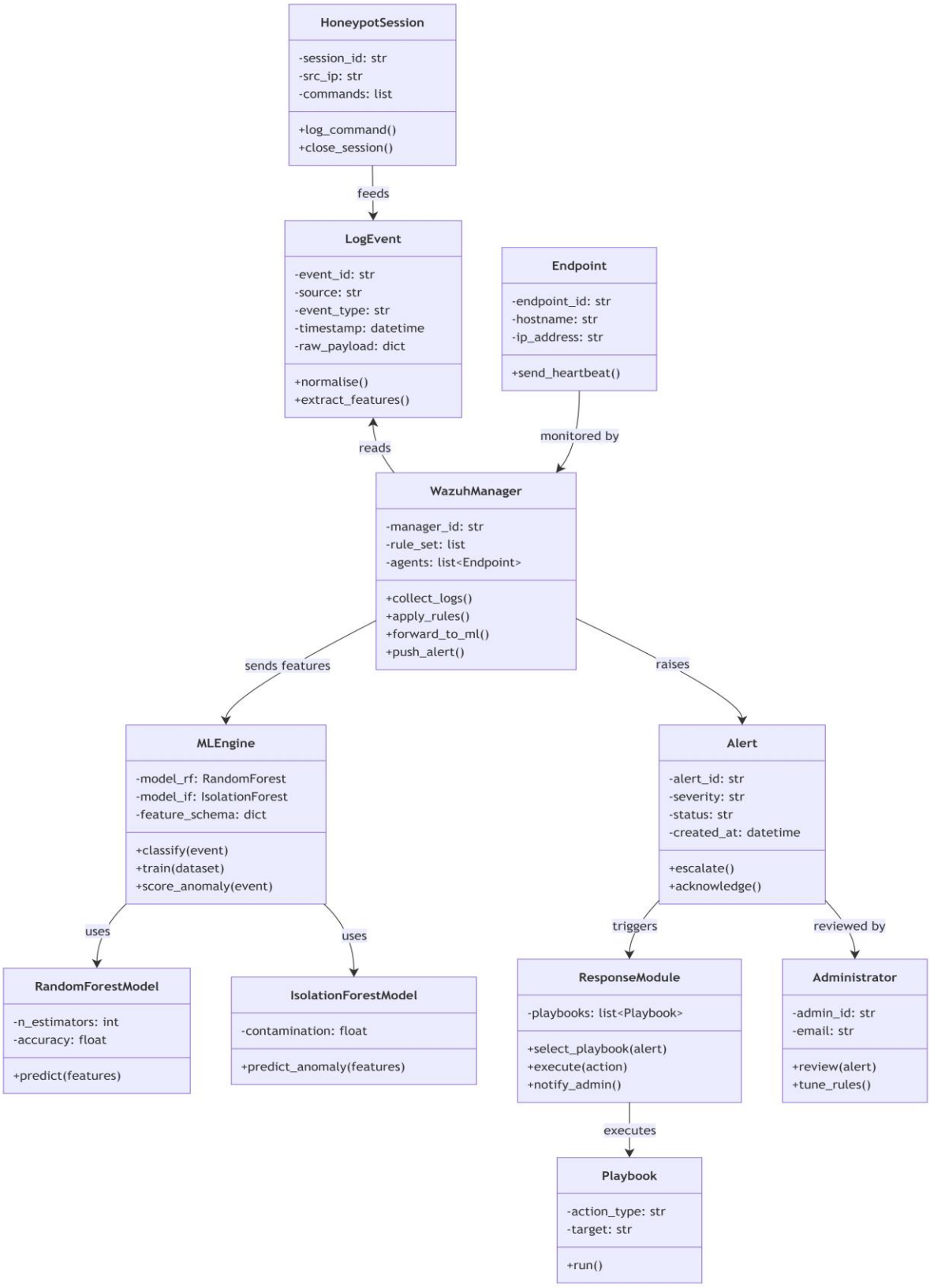



**Figure 4.6: Class Diagram**

The ResponseModule class is triggered by a critical Alert and is composed of one or more Playbook objects, each of which encapsulates a single containment action (IP block, account lock, or email notification) and exposes a uniform run() method, allowing new response actions to be added in future work without modifying the ResponseModule's dispatch logic. The Endpoint and HoneypotSession classes represent the two sources of LogEvent data, and the Administrator class models the human operator who reviews Alerts and tunes the WazuhManager's rule set.

## 4.4 System Design

Building on the analysis in Section 4.3, this section presents the system architecture diagram that specifies how the platform's components are arranged into deployable layers and mapped onto the five-VM VirtualBox test environment described in Section 3.4.

### 4.4.1 System Architecture Diagram

Figure 4.7 shows the platform organised into four horizontal layers, consistent with the four-layer decomposition introduced in Chapter 1. The collection layer, at the bottom, comprises the two monitored endpoints (Endpoint 1 and Endpoint 2, each running Ubuntu Server 22.04 LTS with a Wazuh agent) and the Cowrie honeypot, which together generates the raw event stream. All three components, together with the Kali Linux attack VM, sit inside the isolated VirtualBox internal network boundary, in line with the scope constraint that no component depends on external connectivity.

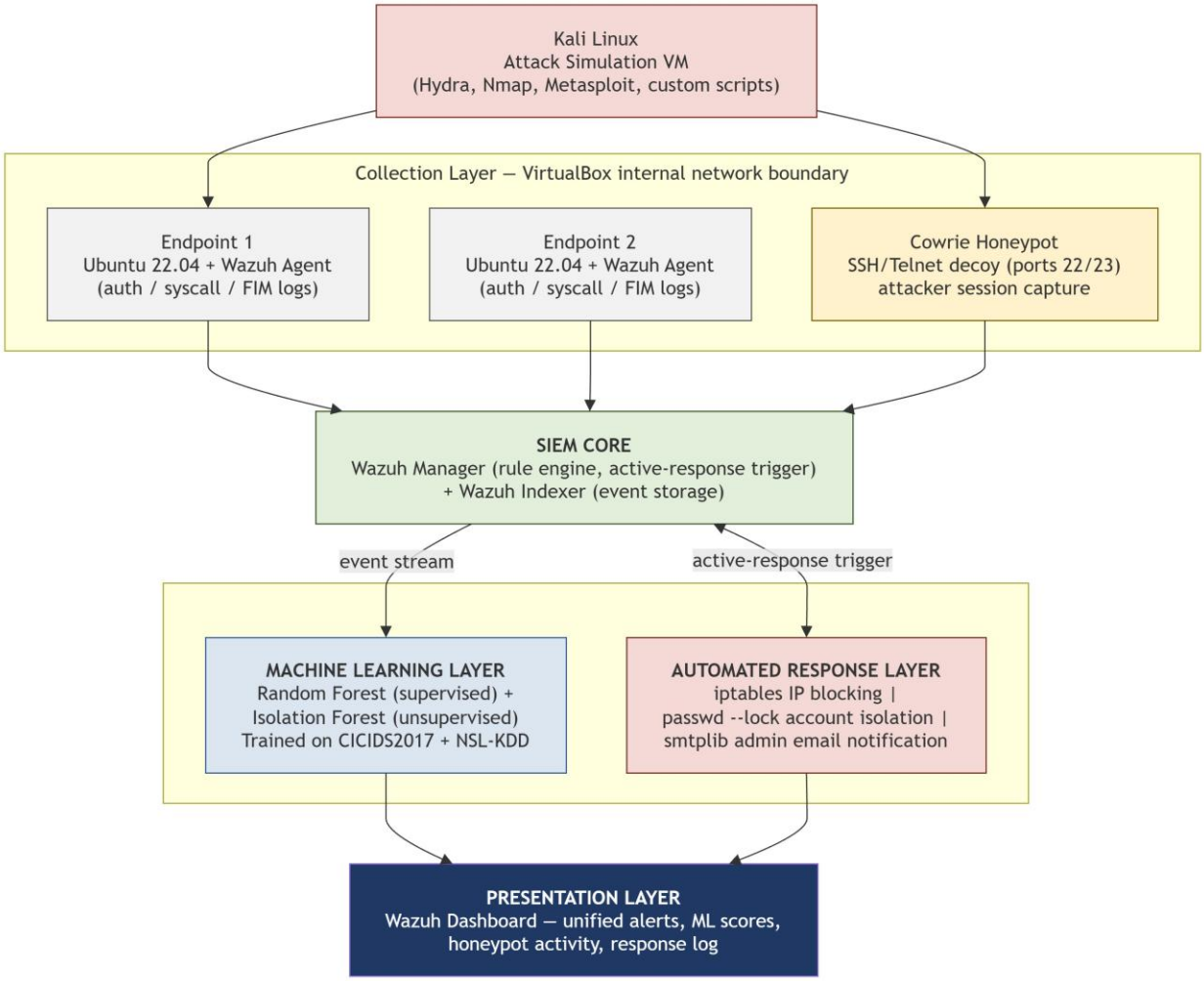



**Figure 4.7: System Architecture Diagram**

Above the collection layer sits the SIEM core, comprising the Wazuh Manager, which applies rule-based detection and hosts the active-response trigger mechanism, and the Wazuh Indexer, which stores normalised events for querying and later analysis. The SIEM core forwards event features upward to the machine learning layer and, on receiving a classification result, forwards a corresponding trigger to the automated response layer. The machine learning layer and the automated response layer are shown side by side because they operate on the same tier of the architecture but perform distinct responsibilities: the former turns raw features into a classification, while the latter turns a classified alert into a containment action, and the two do not communicate directly with each other, only through the SIEM core. At the top of the architecture, the presentation layer, the Wazuh Dashboard, consolidates alerts, ML classification scores, honeypot

session activity, and the response action log into the single real-time view referenced throughout Chapters 1 to 3 as the platform's unified operational interface.

This layered arrangement was chosen over a monolithic design specifically because it isolates the machine learning and automated response components behind the SIEM core: either component can therefore be modified, retrained, or replaced (for example, substituting a different classifier in Sprint 3) without altering how endpoints or the honeypot report events, and without altering how alerts are presented on the dashboard. This separation of concerns directly supports the Agile DSR methodology's requirement, described in Section 3.2, that later sprints can refine individual layers as their requirements become clearer without forcing rework of layers already validated in earlier sprints.